

CS 449 – Lecture 2 Note Template

Dr. Jack Lange

1 Turning work in with tar

This lecture starts with a lab tool, then spends most of its time on a bigger idea: how a C program gets data in and sends data out.

- When would you create a `.tar.gz` versus extract one? What can go wrong if you mix up the flags?
- Write a command you would actually use to bundle this week’s lab files, then the command a TA would use to unpack it.

```
# create:   tar -----  
# extract:  tar -----
```

2 Why C cares about order

The slides contrast C with Java before talking about input. That is the setup: if you do not know how C is parsed, the later code examples will look like magic failures.

- In your own words, what does “C is parsed top down” change about how you write a file?
- Compare C and Java: which kinds of names still have to appear above their use in Java, and which do not? Why might C be stricter?
- What would break if you called a helper function that is defined later in the same `.c` file, with no declaration above `main`?

```
/* Predict: does this compile in C? Why or why not? */
int main(void) {
    foo();
}

void foo(void) {
    /* what would you add or move? */
}
```

3 Three ways a C program gets input

The rest of the lecture is really one question: where can bytes come from, and who is in charge of them?

3.1 Command-line arguments

- Imagine you run `./cmdargs hello 42`. What is in `argc`, `argv[0]`, and `argv[1]`? What is *not* in `argv`?
- When is `argv` a better input tool than typing into the program after it starts?

```
int main(int argc, char *argv[]) {
    // If the user forgot an argument, what should you check?
    // -----
}
```

3.2 Console input with `scanf`

- `scanf` does not just “read text.” What job is the format string doing, and why does the `&` matter?
- A classmate writes `scanf("%d", n);` (no `&`) and the program crashes or silently corrupts memory. What did they misunderstand?

- Why does the lecture care about the return value? Invent a tiny input that should make `scanf` fail.

```
int n;
int matched = scanf("%d", /* what belongs here? */);
/* If matched is 0, what happened, and what is in n? */
```

4 Streams: the idea that ties input and output together

`argv` is a one-shot list. `scanf` is already talking to a stream. The lecture's third mechanism is the general version of that idea.

- Explain FIFO like you are teaching a friend who has never heard the term. Why does “in order” matter for a console?
- What is each of `stdin`, `stdout`, and `stderr` *for*? When would you print an error to `stderr` instead of `stdout`?
- The slides also mention files, devices, and network sockets as streams. What is the shared mental model?

Diagram: sketch a stream from your program, through the OS, to the console (or a file). Mark the buffer and a flush.

Do not copy a figure from memory. Draw the pieces and label who waits if the buffer is not flushed.

4.1 Buffering, printf, and fprintf

- Why can a program look “stuck” or print nothing until it exits, even if you called `printf`?
- `printf` picks a stream for you. `fprintf` makes you pick. When is the extra argument worth it?

```
int printf(const char *format, ...);
int fprintf(FILE *stream, const char *format, ...);
/* Write one situation where fprintf is the better call. */
/* ----- */
```

5 The shell already knows about streams

Unix “everything is a file” is why the same program can talk to a keyboard, a file, or another program without being rewritten.

- In your own words, why can the shell mix files and console streams so easily?

- You ran `ls > files.out` yesterday and today you run `ls >> files.out`. What is different about the file afterward? When is each the one you want?
- Predict what `wc < files.out` does without looking it up from the slide. Who is “typing” into `wc`?
- Walk through `ls | grep .c | wc` as a story: what does each stage receive, and what does it pass on?

6 Special streams the OS leaves lying around

These look like files in the filesystem, but they are features.

- A program is too noisy. You want to keep the program and throw away its output. Which special file do you aim at, and what happens to reads versus writes?
- The slides say `/dev/zero` is different from `/dev/null`. How?
- When might a program read from `/dev/random` instead of those two?

7 When you, not the shell, open the file

Command-line arguments and the three console streams are set up for you. A file you name in code is your problem.

- Why can a text file, once opened, be treated like the console? What functions become the file versions of `printf` / `scanf`?
- What does a `NULL` from `fopen` mean in real life (permissions, bad path, ...)? What should a careful program do next?

```
FILE *file = fopen(name, /* which mode, and why? */);
if (file == NULL) {
    /* what belongs here? */
}
/* talk to the file */
fclose(file);
```

8 What actually lands on disk: ASCII versus raw bytes

This is the lecture’s “aha” about why printing a number and writing a number are not the same.

- What problem is ASCII solving? Why is the character "7" not the same thing as the integer 7 sitting in a variable?
- The slides show `fprintf("%d\n", i)` for $i = 7$ and $i = -7$. Before you copy their hex, predict: why do extra bytes appear in the file that were not the in-memory integer?

Diagram: for one of those examples, show memory on the left and the file on the right. Label which side is “the number” and which side is “text.”

Use the slide values if you want (0x00000007 vs 0x37, 0x0a). The point is the comparison, not the hex trivia.

9 Explicit copies: `fread` and `fwrite`

- The lecture says an application can only operate on data in memory. Why does that force you to copy to and from files yourself?
- File access is “like memory, but explicit.” What is implicit about reading a variable, and what is explicit about `fread`?
- In `fwrite(&x, sizeof(x), 1, outfile)`, what is each argument responsible for? What goes wrong if you pass `x` instead of `&x`?

```
int x = 5;
FILE *outfile = fopen(argv[1], "wb");
fwrite(&x, sizeof(x), 1, outfile);
/* After this, how does the file differ from fprintf("%d", x)? */
fclose(outfile);
```

10 The file position you cannot see

`fread` / `fwrite` do not take an offset. Something else is remembering where you are.

- If a file is an array of bytes, who is keeping the current index, and when does that index move?
- You read 5 bytes, then read again. Where does the second read start? Why can that surprise you?
- What is `fseek(fstream, 16, SEEK_SET)` asking the system to do? Invent a reason you would need that instead of reading from the start every time.

Diagram: draw a file as a row of bytes. Mark the position before a 5-byte read, after it, and after that `fseek`.

Leave room for arrows. The picture should explain the API, not decorate the page.

This template may not cover every topic from the source slides.